PS6 Hackathon Claude Build Prompts

PS6 Hackathon – Claude Build Prompts

Copy-paste prompts for building the PPE, Smoke & Fire Safety AI system, solo, step by step

---

How to use this document

- You are the only builder, working **3-4 hours a day** for **10-12 days**, using a **laptop + Google Colab** for training, plus your **mobile phone as a second camera** later on.
- There are **8 prompts below, one per build phase**, matching your day-by-day plan.
- **Start a new Claude conversation for each phase** and paste in that phase's prompt. Starting fresh each time keeps each conversation focused and keeps you well within usage limits – long single threads burn through your message allowance faster because Claude re-reads the whole thread each time.
- Each prompt is self-contained: it restates the project context, so Claude has everything it needs even in a brand-new chat.
- Work through Claude's response step by step inside that conversation – install, run, fix errors, ask follow-ups – before moving to the next phase's prompt.
- If a phase runs long or something isn't working, just say so in that same conversation ("this step is failing, here's the error: …") rather than starting over.

Phase 1 (Days 1-2): Environment setup + baseline PPE detection model

```
I'm building a solo hackathon project (PS6): an AI system that
detects PPE
(helmet, vest, gloves, boots) on workers, plus smoke/fire, from
camera video,
for a factory safety use case. I have no prior experience with
machine
learning or computer vision -- please explain everything simply and
give me
complete, working code at each step, not just descriptions.

My setup: I have a laptop and access to Google Colab (free GPU). I
have
10-12 days total, working about 3-4 hours a day, doing this entirely
alone.

For this first session, help me:
1. Set up a Google Colab notebook with GPU enabled and confirm it's
working
2. Install the Ultralytics YOLOv8 library
```

```
3. Load a pretrained YOLOv8 model and run it on a sample image/video
just to
   confirm the whole pipeline works end to end
4. Find and download a suitable public PPE detection dataset
(helmet, vest,
   gloves, boots classes) in YOLOv8 format -- recommend a specific
one from
   Roboflow Universe and walk me through downloading it into Colab
5. Fine-tune YOLOv8 on this dataset using transfer learning (not
training
   from scratch)
6. Show me how to read the results (mAP, precision, recall) and tell
me,
   in plain language, whether the numbers I get back are "good
enough to
   move forward" or "needs fixing" -- and if the latter, what to
try.

Walk me through this one step at a time, waiting for me to confirm
each
step worked before giving me the next one.
```

Phase 2 (Days 3-4): Hard-case data, fire/smoke model, zone rules

```
Continuing my solo PS6 hackathon project (PPE + smoke/fire safety AI
for
factories, no prior ML experience, using Colab + laptop, 3-4
hrs/day).

I already have a YOLOv8 model fine-tuned on a public PPE dataset
(helmet,
vest, gloves, boots) from a previous session. Now help me with:

1. What "hard case" photos/clips I should personally capture or find
   (dusty, dark/low-light, angled camera views, someone wearing
bright
   orange/yellow clothing) and how many of each is realistically
enough
   for a solo builder in one session
2. How to label these using Roboflow's free annotation tool and
merge them
   into my existing dataset
3. How to retrain/fine-tune the model with this added data
4. Getting a fire and smoke detector working -- recommend whether I
should
   use an existing pretrained fire/smoke model or fine-tune one
quickly,
   given my time constraints, and give me the exact code either way
5. Writing simple Python logic that, given a detected "person" box
and PPE
   item boxes, decides which PPE items that specific person is
missing
   (not just "a violation exists somewhere in the frame")
6. Creating a simple YAML config file defining 2-3 zones (e.g.
"welding
   bay", "warehouse") with which PPE items are required in each zone

Give me complete code for each part, and explain how they connect
together.
Go one step at a time and wait for my confirmation before
continuing.
```

Phase 3 (Days 5-6): Person tracking + severity engine

```
Continuing my solo PS6 hackathon project (PPE + smoke/fire safety
AI, no
prior ML experience, Colab + laptop, 3-4 hrs/day). I now have a
fine-tuned
PPE model, a fire/smoke detector, per-person item-association logic,
and a
zone rules config from previous sessions.

Help me build the next layer:

1. Add multi-object tracking using ByteTrack via the Supervision
library,
   so each worker gets a persistent ID across video frames --
explain why
   this matters (avoiding duplicate alerts) and give me working code
2. Build a simple state machine per (worker ID, PPE item) that only
raises
   an alert if the item is missing in most of the last several
frames
   (not just one frame), waits a short grace period before alerting,
and
   has a cooldown so the same violation doesn't spam alerts
3. Build a severity engine with 4 levels (Critical, High, Medium,
Low) that
   assigns severity based on event type and the zone's risk level --
e.g.
   fire is always Critical, a missing helmet in a high-risk zone is
High,
   a missing glove in a low-risk zone is Medium
4. Show me how to test this on a sample video and confirm I'm
getting one
   alert per real violation, not dozens

If the ByteTrack/Supervision library gives me setup trouble, tell me
immediately and give me a simpler fallback (a basic tracker) instead
of
letting me get stuck -- I only have a few hours for this phase.

Go one step at a time, waiting for my confirmation before
continuing.
```

Phase 4 (Days 7-8): Streamlit dashboard + running multiple camera sources

```
Continuing my solo PS6 hackathon project (PPE + smoke/fire safety
AI, no
prior ML experience, Colab + laptop, 3-4 hrs/day). I now have a
working
detection + tracking + severity pipeline from previous sessions that
produces alerts (worker ID, missing item, zone, severity,
confidence,
timestamp).

Help me build a dashboard using Streamlit (not React -- I need
something
fast to build alone):

1. A live view tile showing the annotated video feed
2. A live alert feed list, color-coded by severity, showing zone,
camera,
```

```
   violation type, timestamp, and confidence
3. Acknowledge / dismiss buttons on each alert that update its
status
4. A simple table showing the zone rules (which PPE is required
where)
5. Store alerts in a local SQLite database so they persist

Then help me run this against TWO video sources at once:
- One looped recorded video file (acting as "Camera 1")
- My laptop's webcam (acting as "Camera 2")
Each with a visible online/offline status indicator on the
dashboard.

Give me complete, working code, explain how the pieces connect, and
go
one step at a time, confirming each part works before moving to the
next.
```

Phase 5 (Day 9): Adding my phone as a live camera

```
Continuing my solo PS6 hackathon project. I have a working Streamlit
dashboard running detection on a recorded video file and my laptop
webcam
as two camera sources (from a previous session).

Now I want to add my Android phone as a third, live camera source
using
an app like IP Webcam or DroidCam. Help me:

1. Decide which app is easier for a beginner and walk me through
setting
   it up on my phone
2. Get the phone's video stream URL and connect it into my existing
   pipeline as a new camera source, using the same code structure as
my
   webcam source
3. Confirm the phone stream produces alerts in the exact same format
as
   my other two sources -- no special-casing
4. Troubleshoot common issues (Wi-Fi connection between phone and
laptop,
   stream lag, connection drops)

Give me complete code for the integration and go one step at a time.
```

Phase 6 (Day 10): Robustness testing + failure-case gallery

```
Continuing my solo PS6 hackathon project. I now have all 3 camera
sources
(recorded video, webcam, phone) feeding into my detection +
dashboard
pipeline from previous sessions.

Help me stress-test it before the hackathon:

1. Give me a checklist of specific test clips/scenarios I should run
   through the system: dusty/hazy footage, low light, an angled
camera
   view, and -- most importantly -- someone wearing a bright
orange/yellow
```

```
   vest walking through frame (to confirm it does NOT falsely
trigger a
   fire alert)
2. For each test, tell me what a "correct" result looks like versus
a
   "something's wrong" result
3. Help me interpret and fix whatever specific issues come up as I
run
   these (I'll paste you what I'm seeing)
4. Show me how to capture and organize before/after screenshots of
each
   test case into a simple folder structure I can use for a
"failure-case
   gallery" slide later

Let's go through this test by test -- I'll tell you the result of
each one
and you tell me the next test or the fix.
```

Phase 7 (Day 11): Dashboard polish, reports, and a backup demo video

```
Continuing my solo PS6 hackathon project. My detection pipeline and
dashboard are working and tested across all 3 camera sources from
previous sessions.

Help me finish the last technical stretch before the pitch:

1. Polish the dashboard's visual design so it looks clean and
   professional (spacing, colors, layout) -- give me the updated
code
2. Add a simple "violations by zone" bar chart using the alert data
   already stored in SQLite
3. Add a button to export the alert log as a CSV file
4. Walk me through recording a full backup demo video of the system
   working end to end, in case my live camera fails during the
actual
   hackathon presentation
5. Help me tag this working version clearly (e.g. in git or just a
   clearly labeled folder copy) so I always have something stable to
   fall back to

Give me complete code for the dashboard changes and go one step at a
time.
```

Phase 8 (Day 12): Pitch deck, rehearsal, final fixes

```
Continuing my solo PS6 hackathon project. My system (detection,
tracking,
severity, dashboard, 3 camera sources) is built and tested from
previous
sessions, and I have failure-case screenshots and a backup demo
video.

Help me finish the submission:

1. Build me a 10-slide pitch deck outline covering: the problem, the
gap
   in existing solutions, my architecture (edge-first, zone-aware,
   severity-tiered), a live demo script, my robustness/failure-case
   evidence, measured results, an honest "what's mocked or
```

```
simplified"
   slide, and a short roadmap
2. Write draft slide content for each one in plain language, based
on
   what I've actually built (not features I don't have)
3. Give me a rehearsed demo script: exact order of actions to
perform live
   (which camera to show first, when to trigger a violation, when to
show
   the fire false-positive case) so it flows smoothly in front of
judges
4. Give me a short list of likely judge questions or edge case
issues I should double check and rehearse for, so I'm not caught off
guard

I only want fixes at this point, not new features -- help me stay
focused
on finishing cleanly rather than adding anything new.
```

Quick tips for using these efficiently

- Paste the whole prompt for a phase in one message rather than splitting it into several small messages – this uses your usage allowance more efficiently.
- Keep each phase in its own conversation. When a conversation is done, move to the next phase’s prompt in a new chat.
- If you hit a usage limit mid-phase, it resets in a few hours – just resume in the same conversation once it does.
- Paste in error messages, screenshots, or results exactly as you see them – the more specific you are, the less back-and-forth is needed.